

Examen 1. TITANIC

Ing. Isaias Garcia Zanella

17 de abril de 2026

1. Introducción

El presente informe detalla el desarrollo e implementación de un modelo de clasificación basado en el algoritmo de K-Vecinos más Cercanos (KNN) para predecir la supervivencia de los pasajeros del Titanic. El análisis se llevó a cabo utilizando dos enfoques de evaluación: la técnica de Hold-out y la validación cruzada de K-pliegues, con el objetivo de comparar su desempeño y obtener una estimación robusta de la capacidad predictiva del modelo y aplicar todo lo aprendido en clase durante el curso de Machine Learning.

2. Marco Teórico

2.1. Aprendizaje Máquina (Machine Learning)

Tradicionalmente, el diccionario define «aprender» de varias maneras, todas ellas relacionadas a procesos humanos de adquisición de conocimientos.

1. Obtener conocimiento ya sea por estudio, experiencia o enseñanza.
2. Ser consciente o darse cuenta de cierta información a través de la observación.
3. Recibir instrucción de manera formal o informal.
4. Ser informado acerca de algo.

No obstante, cuando trasladamos el concepto al ámbito de los sistemas computacionales, surgen ciertas limitaciones y desafíos semánticos.

Sin embargo, una interpretación pragmática del aprendizaje de máquina (machine learning), dentro del campo de la inteligencia artificial, puede entenderse como la habilidad de un sistema para mejorar su desempeño en una tarea específica mediante la experiencia, sin ser explícitamente programado para ello. Aquí la « experiencia » se refiere al uso de datos —conocidos como datos de entrenamiento— que permiten al sistema identificar patrones, regularidades y relaciones, a partir de las cuales se construyen modelos predictivos o descriptivos. [3].

Este proceso de aprendizaje se fundamenta en la optimización de una función objetivo, donde el algoritmo ajusta sus parámetros internos para minimizar el error entre sus predicciones y la realidad observada. En este contexto, el algoritmo no solo almacena información,

sino que desarrolla la capacidad de inducción, permitiéndole tomar decisiones sobre datos que nunca ha procesado anteriormente [1].

2.1.1. Clasificación del aprendizaje en Inteligencia Artificial

En la práctica, el aprendizaje máquina se clasifica desde diferentes perspectivas, principalmente:

- Según la forma en que el modelo aprende los datos:
 - Aprendizaje supervisado: El sistema aprende a partir de un conjunto de datos etiquetados, donde cada ejemplo de entrenamiento incluye tanto las características de entrada como la salida deseada. El objetivo es que el modelo generalice a partir de estos ejemplos para predecir la salida correcta para nuevos datos no vistos.
 - Aprendizaje no supervisado: En este caso, el sistema trabaja con datos sin etiquetar, buscando patrones, estructuras o agrupaciones dentro de los datos. El objetivo es descubrir relaciones ocultas o segmentar los datos en grupos similares.
 - Aprendizaje por refuerzo: Aquí, el sistema aprende a través de la interacción con un entorno, recibiendo recompensas o castigos en función de sus acciones. El objetivo es maximizar la recompensa acumulada a lo largo del tiempo.
- Según el estilo o enfoque del aprendizaje:
 - Aprendizaje inductivo: Generaliza reglas o patrones a partir de ejemplos específicos.
 - Aprendizajes deductivos: Aplica reglas generales para inferir resultados específicos.
 - Aprendizaje transductivo: No generaliza, sino que predice valores directamente para los casos desconocidos durante el aprendizaje.
 - Aprendizaje en línea vs. por lotes: El aprendizaje puede hacerse de forma continua conforme llegan los datos (en línea) o usando todo el conjunto de datos de una sola vez (por lotes).

[3]

Cabe destacar que la elección entre estos enfoques depende directamente de la naturaleza del problema y la disponibilidad de etiquetas. Por ejemplo, en problemas de clasificación binaria como la predicción de supervivencia, el aprendizaje supervisado es el estándar, mientras que el análisis de perfiles de clientes suele abordarse mediante el aprendizaje no supervisado [4].

Al trabajar con algoritmos de aprendizaje automático, uno de los primeros pasos críticos es la identificación y comprensión de los tipos de datos disponibles. En ciencia de datos, los atributos o variables pueden clasificarse en diversas escalas de medición:

- Datos categóricos: Representan categorías sin orden inherente, como el género o la profesión.

- Datos ordinales: Similares a los nominales, pero con un orden específico, como el nivel de educación o la satisfacción del cliente.
- Datos de intervalo: Tienen un orden y distancias significativas entre valores, pero sin un cero absoluto, como la temperatura en Celsius.
- Datos de proporción: Similares a los de intervalo, pero con un punto cero absoluto (por ejemplo, peso, edad, ingresos).

La correcta identificación de estas escalas permite determinar el preprocesamiento adecuado, como la aplicación de técnicas de codificación (*one-hot encoding*) para datos categóricos o la normalización para datos de proporción, asegurando que el modelo no asuma erróneamente un orden o importancia superior debido a la magnitud de las escalas numéricas [5].

Otro aspecto fundamental en la construcción de modelos de aprendizaje automático es la división de los datos en diferentes subconjuntos, siendo los principales:

- Conjunto de entrenamiento: Utilizado para ajustar el modelo; es decir, para que este aprenda los patrones, relaciones y estructuras contenidas en los datos.
- Conjunto de prueba (o validación): Empleado para evaluar la capacidad del modelo para generalizar a datos no vistos durante el entrenamiento. Sirve para estimar el rendimiento real del modelo y ajustar hiperparámetros.

[3]

Para garantizar una evaluación robusta y mitigar el sesgo introducido por una única partición aleatoria, se recomienda el uso de técnicas como la validación cruzada de K-pliegues (*K-Fold Cross Validation*). Este método permite que cada dato del conjunto participe tanto en el entrenamiento como en la validación, proporcionando una estimación más precisa del desempeño del modelo en el mundo real [7].

2.2. Sobreajuste

El sobreajuste o sobreaprendizaje ocurre cuando un modelo ajusta sus parámetros de manera excesiva a los datos de entrenamiento, capturando no solo los patrones verdaderos, sino también el ruido, outliers o irregularidades del conjunto. Esto se manifiesta cuando el modelo tiene una precisión muy alta en el entrenamiento, pero pobre desempeño en datos de prueba, lo que implica que no generaliza. Las causas más comunes para esto son:

- Modelos excesivamente complejos.
- Falta de regularización.
- Entrenamiento prolongado sin validación cruzada.
- Conjuntos de datos pequeños o desequilibrados.

Algunas soluciones más utilizadas para lidiar con el sobreajuste incluyen:

- Regularización (L1, L2).
- Reducción de la complejidad del modelo.
- Aumento del conjunto de datos.
- Validación cruzada.
- Técnicas de ensamble.

[3]

2.3. Subajuste

Por el contrario, el subajuste o subaprendizaje ocurre cuando el modelo no logra aprender correctamente los patrones de los datos, ni siquiera en el conjunto de entrenamiento. Esto significa que el modelo es demasiado simple o está mal configurado para capturar la complejidad del problema. Algunas de las causas más comunes son las siguientes:

- Modelos muy simples (como una regresión lineal para datos no lineales).
- Insuficiente entrenamiento (muy pocas épocas o iteraciones).
- Hiperparámetros mal seleccionados.
- Transformación inadecuada de los datos.

Algunas soluciones más utilizadas para lidiar con el subajuste incluyen:

- Aumentar la complejidad del modelo.
- Mejorar la ingeniería de características.
- Aumentar el tiempo de entrenamiento.
- Revisar los supuestos del modelo frente a la naturaleza de los datos.

[3]

2.4. Aprendizaje Basado en Similitud

El aprendizaje basado en similitud en *machine learning* fundamenta su paradigma en el principio heurístico de que los patrones históricos exitosos constituyen predictores confiables para instancias futuras. Es decir, este enfoque parte de la premisa de que la mejor manera de realizar una predicción o clasificación es identificar instancias pasadas con características similares a la consulta actual y proyectar un comportamiento análogo.

A diferencia de los modelos basados en parámetros que intentan resumir los datos en una función matemática, los modelos de similitud mantienen una relación directa con las instancias individuales del conjunto de datos. El modelo de aprendizaje máquina basado en similitud más popular es el algoritmo de K-Vecinos más cercanos (*K-Nearest Neighbors* o

KNN). Este algoritmo es ampliamente reconocido por su simplicidad y eficacia en tareas de clasificación y regresión [1].

El algoritmo KNN implementa un enfoque de aprendizaje perezoso (*lazy learning*), donde la fase de entrenamiento se reduce esencialmente a almacenar el conjunto completo de instancias etiquetadas $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$. Al no generar un modelo discriminativo explícito durante el entrenamiento, el costo computacional se traslada a la fase de inferencia. Durante esta etapa, para una nueva instancia de consulta (x_{query}), el algoritmo ejecuta los siguientes pasos: [4].

1. **Cálculo de distancia:** Computa la métrica de proximidad $d(x_{query}, x_i)$ para todos los puntos del conjunto de entrenamiento. Las métricas más comunes incluyen la distancia Euclidiana, la distancia de Manhattan y la distancia de Minkowski.
2. **Selección de vecinos:** Identifica los K puntos más cercanos a x_{query} basándose en las distancias calculadas.
3. **Votación o promedio:** Realiza la clasificación mediante una votación mayoritaria entre las etiquetas de los K vecinos, o una predicción de valor continuo mediante el promedio de los mismos en casos de regresión.

La efectividad del algoritmo KNN depende críticamente de dos factores: la elección del valor K y la métrica de distancia empleada. Un valor de K demasiado pequeño puede hacer que el modelo sea sensible al ruido (sobreajuste), mientras que un valor demasiado grande puede suavizar excesivamente las fronteras de decisión, ignorando patrones locales importantes (subajuste). Asimismo, dado que KNN se basa en distancias, es fundamental realizar una normalización o estandarización previa de las características para evitar que aquellas con magnitudes mayores dominen el cálculo de la similitud [7].

[3]

2.5. Métricas de evaluación

La evaluación del desempeño de un modelo de clasificación es un paso crítico para determinar su capacidad de generalización. La base fundamental para el cálculo de la mayoría de estas métricas es la *Matriz de Confusión*, una tabla que desglosa las predicciones del modelo en cuatro categorías: Verdaderos Positivos (VP), Verdaderos Negativos (VN), Falsos Positivos (FP) y Falsos Negativos (FN) [2].

A partir de esta matriz, se derivan las siguientes métricas de evaluación:

- **Exactitud (Accuracy):** Es la proporción de predicciones correctas (tanto positivas como negativas) respecto al total de casos. Aunque es la métrica más intuitiva, puede ser engañosa en conjuntos de datos desequilibrados.

$$\text{Accuracy} = \frac{VP + VN}{VP + VN + FP + FN}$$

- **Precisión (Precision):** Indica la calidad de las predicciones positivas. Es la fracción de casos identificados como positivos que realmente lo son. Es crucial cuando el costo de un Falso Positivo es alto.

$$\text{Precision} = \frac{VP}{VP + FP}$$

- **Sensibilidad (Recall / Exhaustividad):** Mide la capacidad del modelo para detectar todos los casos positivos reales. En el contexto del Titanic, un *recall* alto significaría que el modelo no .olvida.^a los sobrevivientes reales.

$$\text{Recall} = \frac{VP}{VP + FN}$$

- **Especificidad:** Representa la tasa de verdaderos negativos; es decir, la capacidad del modelo para identificar correctamente los casos de la clase negativa (quienes no sobrevivieron).

$$\text{Especificidad} = \frac{VN}{VN + FP}$$

- **Puntaje F-Beta (F_β):** Es una generalización del F1-Score que permite ponderar la importancia de la precisión frente al *recall* mediante un parámetro β . Si $\beta = 1$, obtenemos el F1-Score (media armónica balanceada). Si $\beta > 1$, se le da más importancia a la sensibilidad.

$$F_\beta = (1 + \beta^2) \cdot \frac{\text{Precision} \cdot \text{Recall}}{(\beta^2 \cdot \text{Precision}) + \text{Recall}}$$

- **Área bajo la Curva ROC (AUC-ROC):** La curva ROC (*Receiver Operating Characteristic*) representa la tasa de verdaderos positivos frente a la tasa de falsos positivos. El AUC proporciona un valor agregado que mide la capacidad del modelo para distinguir entre clases independientemente del umbral de decisión [4].

La elección de la métrica adecuada depende del objetivo del análisis. En presencia de conjuntos de datos con clases minoritarias, métricas como el F-Score o el AUC suelen ser preferibles sobre la exactitud simple, ya que proporcionan una visión más holística del equilibrio entre errores de Tipo I y Tipo II [6].

2.6. Validación Cruzada de K-Pliegues (K-Fold Cross Validation)

La validación cruzada de K-pliegues es una técnica robusta de evaluación de modelos que busca mitigar el sesgo introducido por una partición única de los datos. En lugar de depender de una sola división de entrenamiento y prueba, el conjunto de datos se divide en K subconjuntos o "pliegues" de tamaño aproximadamente igual [5].

El proceso se ejecuta de manera iterativa K veces, siguiendo esta lógica:

1. En cada iteración i , el pliego k_i se reserva para la validación del modelo.
2. Los restantes $K - 1$ pliegues se utilizan para la fase de entrenamiento.

3. Se calcula la métrica de desempeño (como el *accuracy* o el error) para dicha iteración.

Al finalizar las iteraciones, el rendimiento del modelo se estima mediante el promedio de las métricas obtenidas en cada pliegue. Este enfoque garantiza que cada instancia del dataset participe tanto en el entrenamiento como en la prueba, proporcionando una estimación de la precisión mucho más estable y confiable, especialmente en conjuntos de datos de tamaño limitado como el del Titanic [7].

2.7. Método del Codo (Elbow Method) para la Optimización de K

En el algoritmo KNN, la elección del hiperparámetro K (número de vecinos) es determinante para el equilibrio entre el sesgo y la varianza. El método del codo es una técnica heurística utilizada para identificar el valor óptimo de K mediante el análisis de la tasa de error o la precisión en función de diferentes valores de vecindad.

El procedimiento consiste en graficar el desempeño del modelo frente a un rango de valores de K (por ejemplo, de 1 a 31). A medida que K aumenta, la curva de error tiende a descender inicialmente; sin embargo, existe un punto donde la mejora en el rendimiento se vuelve marginal o comienza a degradarse debido al sobreajuste o subajuste.

Este punto de inflexión en la curva, donde el cambio de pendiente es más pronunciado, se denomina "codo". Seleccionar el valor de K en esta región permite obtener un modelo que generaliza correctamente, evitando la complejidad innecesaria de un K muy grande y la sensibilidad al ruido de un K extremadamente pequeño [8].

3. Materiales y Métodos

La base de datos con la que vamos a trabajar es el conjunto de datos del Titanic, disponible en la plataforma Kaggle. Este dataset contiene información detallada sobre los pasajeros del Titanic, incluyendo características como edad, género, clase de boleto, tarifa pagada, entre otras, así como la variable objetivo que indica si el pasajero sobrevivió o no al desastre. El análisis se llevará a cabo utilizando Python, con bibliotecas especializadas en ciencia de datos como *pandas* para la manipulación de datos y la evaluación del modelo, y *matplotlib* para la visualización de resultados. El proceso metodológico incluirá las siguientes etapas:

- **Preprocesamiento de datos:** Limpieza de datos, manejo de valores faltantes, codificación de variables categóricas y normalización de características numéricas.
- **División del dataset:** Separación del conjunto de datos en entrenamiento y prueba, utilizando una proporción estándar (por ejemplo, 80 % para entrenamiento y 20 % para prueba).
- **Entrenamiento del modelo:** Implementación del algoritmo KNN utilizando clases, con una búsqueda de hiperparámetros para determinar el valor óptimo de K mediante el método del codo.
- **Evaluación del modelo:** Cálculo de métricas.

- **Validación cruzada:** Aplicación de la validación cruzada de K-pliegues para obtener una estimación más robusta del desempeño del modelo.
- **Visualización de resultados:** Creación de gráficos para ilustrar la relación entre el valor de K y las métricas de desempeño, así como la matriz de confusión para evaluar la distribución de predicciones.

Ahora pasaremos a explicar algunas funciones que se utilizarán en el código para la implementación del modelo KNN y la evaluación de su desempeño. Cabe mencionar que se utilizaron algunas funciones previamente creadas para la codificación y normalización de datos en algunas columnas, esto para que el modelo pueda tener una mayor eficacia al realizar el entrenamiento, comentado esto ahora si continuaremos con la implementación del algoritmo KNN.

3.1. Implementación del Algoritmo KNN

Para el desarrollo de este proyecto, se optó por una implementación propia del clasificador de K-Vecinos más Cercanos dentro del entorno de Python. La arquitectura de la clase `KNeighbors_Classifier` sigue el paradigma de programación orientada a objetos (POO) y se estructura bajo los tres pilares fundamentales del algoritmo: almacenamiento de datos (*Lazy Learning*), cálculo de distancias y votación mayoritaria.

3.1.1. Descripción de la Clase y Métodos

- **Constructor (`__init__`):** Define el hiperparámetro k , que representa el número de vecinos a considerar para la clasificación.
- **Método `fit`:** A diferencia de otros modelos, el entrenamiento en KNN es pasivo. En este método se realiza la asignación del conjunto de entrenamiento a la memoria del objeto, almacenando las dimensiones de la matriz de características.
- **Método `euclidean`:** Implementa la métrica de distancia euclidiana entre dos vectores utilizando la norma L^2 . Esta función es el núcleo geométrico del algoritmo.
- **Método `find_neighbors`:** Ejecuta el cálculo de distancias desde un punto de consulta hacia todas las instancias de entrenamiento. Posteriormente, utiliza `argsort` para obtener los índices de las k distancias más pequeñas y retorna sus etiquetas correspondientes.
- **Método `predict`:** Orquesta la inferencia para múltiples instancias de prueba. Para cada dato, identifica sus vecinos y aplica la función estadística `mode` (moda) para determinar la clase ganadora por mayoría de votos.

3.1.2. Código Fuente del Clasificador

Listing 1: Implementación manual del clasificador KNN en Python

```
1 class KNeighbors_Classifier():
2     def __init__(self, k):
3         self.k = k
4
5     def fit(self, X_train, Y_train):
6         self.X_train = X_train
7         self.Y_train = Y_train
8         self.m, self.n = X_train.shape
9
10    def predict(self, X_test):
11        m_test = X_test.shape[0]
12        Y_predict = np.zeros(m_test)
13        for i in range(m_test):
14            neighbors_labels = self.find_neighbors(X_test[i])
15            # Se calcula la moda para la votacion mayoritaria
16            m = mode(neighbors_labels, keepdims=True)
17            Y_predict[i] = m.mode.flatten()[0]
18
19        return Y_predict
20
21    def find_neighbors(self, x):
22        euclidean_distances = np.zeros(self.m)
23        for i in range(self.m):
24            d = self.euclidean(x, self.X_train[i])
25            euclidean_distances[i] = d
26
27        # Obtener indices que ordenan las distancias de menor a mayor
28        inds = euclidean_distances.argsort()
29        # Ordenamos las etiquetas segun la cercania
30        Y_train_sorted = self.Y_train[inds]
31        return Y_train_sorted[:self.k]
32
33    def euclidean(self, x, x_train):
34        # Implementacion matematica de la distancia euclidiana
35        return np.sqrt(np.sum(np.square(x - x_train)))
```

3.2. Estrategias de Partición y Validación de Datos

Para evaluar la capacidad de generalización del clasificador KNN, se implementaron dos estrategias distintas de partición de datos. El objetivo de este enfoque dual es contrastar una evaluación estática frente a una validación dinámica y exhaustiva.

3.2.1. División Estática de Datos (Hold-out Validation)

La primera aproximación para la evaluación del modelo consiste en el método de *Hold-out*, el cual separa el conjunto de datos en dos bloques independientes. Esta técnica permite entrenar el algoritmo en una porción mayoritaria de la muestra y reservar un porcentaje menor para validar su desempeño final, garantizando que el modelo no sea evaluado con los mismos datos que conoció durante el entrenamiento.

Para asegurar que los resultados sean reproducibles, se implementa una semilla aleatoria (*random seed*). Esto es fundamental en el ámbito académico, ya que permite que otros investigadores obtengan la misma partición exacta de datos.

Listing 2: Función para la separación estática de datos

```
1 def separar_datos(df, objetivo, test_size=0.2, random_state=42):
2     # Definir una semilla para que los resultados sean reproducibles
3     np.random.seed(random_state)
4
5     # Mezclar los indices del DataFrame
6     indices = df.index.tolist()
7     np.random.shuffle(indices)
8
9     # Calcular cuantos datos van para test
10    tamano_test = int(len(df) * test_size)
11
12    # Dividir los indices
13    indices_test = indices[:tamano_test]
14    indices_train = indices[tamano_test:]
15
16    # Crear los sets de entrenamiento y prueba
17    train_df = df.loc[indices_train]
18    test_df = df.loc[indices_test]
19
20    # Separar en X (caracteristicas) y Y (lo que queremos predecir)
21    X_train = train_df.drop(columns=[objetivo]).values
22    Y_train = train_df[objetivo].values
23    X_test = test_df.drop(columns=[objetivo]).values
24    Y_test = test_df[objetivo].values
25
26    return X_train, X_test, Y_train, Y_test, indices_test
```

3.2.2. Validación Cruzada (K-Fold Cross Validation)

Como método de validación avanzada, se implementó la técnica de *K-Fold Cross Validation*. A diferencia de la partición estática, este procedimiento divide el dataset en K subconjuntos o pliegues. El proceso se repite K veces, rotando en cada iteración el pliegue que funciona como conjunto de prueba.

La función `validacion_k_fold_retornar_modelo` ha sido diseñada no solo para calcular la estabilidad del algoritmo, sino para identificar y extraer la instancia del modelo que presenta el mejor desempeño (*accuracy*) histórico. Esto permite mitigar el riesgo de sesgo por selección y proporciona una visión integral de la capacidad predictiva del sistema sobre toda la población de datos disponible.

Listing 3: Función para validación cruzada y selección del mejor modelo

```

1 def validacion_k_fold_retornar_modelo(df, objetivo, k_folds=5, k_vecinos=5):
2     indices = df.index.tolist()
3     np.random.seed(42)
4     np.random.shuffle(indices)
5
6     folds_indices = np.array_split(indices, k_folds)
7
8     mejor_accuracy = 0
9     mejor_modelo = None
10    mejores_indices_test = None
11    lista_accuracy = []
12
13    for i in range(k_folds):
14        indices_test = folds_indices[i]
15        indices_train = np.concatenate([folds_indices[j] for j in range(k_folds) if j != i])
16
17        # Separacion de datos
18        train_df = df.loc[indices_train]
19        test_df = df.loc[indices_test]
20
21        X_train = train_df.drop(columns=[objetivo]).values
22        Y_train = train_df[objetivo].values
23        X_test = test_df.drop(columns=[objetivo]).values
24        Y_test = test_df[objetivo].values
25
26        # Entrenar modelo
27        modelo_actual = KNeighborsClassifier(k=k_vecinos)
28        modelo_actual.fit(X_train, Y_train)
29
30        # Validar
31        predicciones = modelo_actual.predict(X_test)
32        accuracy_actual = np.mean(predicciones == Y_test)
33        lista_accuracy.append(accuracy_actual)
34
35        # GUARDAR EL MEJOR
36        if accuracy_actual > mejor_accuracy:
37            mejor_accuracy = accuracy_actual
38            mejor_modelo = modelo_actual
39            mejores_indices_test = indices_test
40
41    return mejor_modelo, mejores_indices_test, lista_accuracy

```

3.3. Optimización del Hiperparámetro K (Curva de Desempeño)

La selección del valor óptimo de K es un proceso empírico que busca maximizar la capacidad predictiva del modelo. Un valor de K muy bajo puede conducir a un sobreajuste (*overfitting*) debido a la sensibilidad al ruido, mientras que un K excesivamente elevado puede generar un subajuste (*underfitting*) al ignorar estructuras locales en los datos.

Para determinar el valor ideal, se implementó un proceso de iteración sistemática sobre un rango de valores de $K \in [1, 20]$. En cada iteración, se entrena el modelo y se evalúa su desempeño frente al conjunto de prueba, registrando la tasa de acierto (*accuracy rate*). Los

resultados se visualizan mediante una gráfica de línea que permite identificar el "punto de codo." la zona de mayor estabilidad.

Listing 4: Algoritmo de búsqueda del valor K óptimo y visualización de resultados

```
1 range_used = range(1, 21)
2 acierto_rate = []
3
4 for i in range_used:
5     # Se instancia el modelo con el valor actual de K
6     knn = KNeighborsClassifier(k=i)
7     knn.fit(X_train, Y_train)
8     pred_i = knn.predict(X_test)
9
10    # Se calcula la tasa de acierto y se almacena
11    acierto_rate.append(np.mean(pred_i == Y_test))
12
13    # Graficamos los resultados para analisis visual
14    plt.figure(figsize=(10,6))
15    plt.plot(range_used, acierto_rate,
16            color='blue',
17            linestyle='dashed',
18            marker='o', markerfacecolor='red', markersize=10)
19
20    # Ajuste de etiquetas en el eje X para mayor precision
21    plt.xticks(range_used)
22    plt.title('Tasa de Acierto vs. Valor de K')
23    plt.xlabel('K')
24    plt.ylabel('Tasa de Acierto')
25    plt.show()
```

4. Resultados y Discusión

Antes de empezar a entrenar el modelo debemos de observar primero como se ve de forma visual las personas que sobrevivieron y las que no, esto para tener una idea de como se ve el dataset y así poder identificar patrones visuales que puedan ayudar al modelo a realizar una mejor predicción.



Figura 1: Distribución de sobrevivientes vs no sobrevivientes en función de la edad y clase Social.

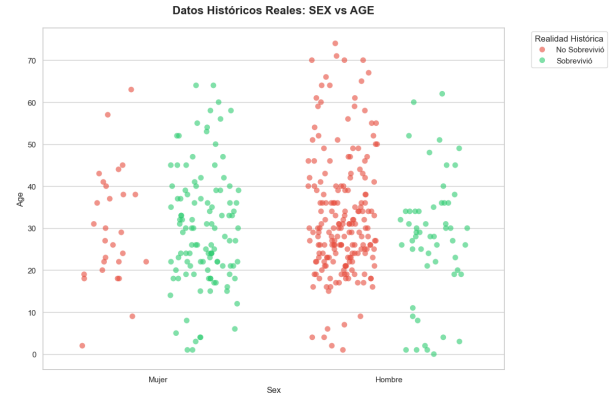


Figura 2: Distribución de sobrevivientes vs no sobrevivientes en función de la edad y Genero.

Con esto podemos observar que la mayoría de los sobrevivientes eran mujeres y niños, esto se puede observar en la grafica de la izquierda, donde se muestra que las personas de clase social mas alta tenian una mayor tasa de supervivencia, y en la grafica de la derecha se observa que las mujeres tenian una mayor tasa de supervivencia que los hombres, esto es un patron visual que el modelo puede identificar para realizar una mejor prediccion.

Una vez observado esto, ahora se procederá a presentar algunos de los métodos utilizados para la implementación del modelo KNN y la evaluación de su desempeño, se procederá a presentar los resultados obtenidos a partir de la aplicación de estas técnicas sobre el dataset del Titanic. Se analizarán las métricas de desempeño, la curva de acierto en función del valor de K , y se discutirá la interpretación de estos resultados en el contexto del problema planteado.

4.1. Primera prueba.- KNN y Hold-out Validation

Despues de discriminar las columnas que no aportaban informacion relevante para el modelo, se procedio a realizar la implementacion del algoritmo KNN utilizando la tecnica de Hold-out para la particion de datos. Y con ello se realizo 20 veces, esto para obtener una curva de acierto en funcion del valor de K , y asi poder identificar el valor optimo de K para nuestro modelo.

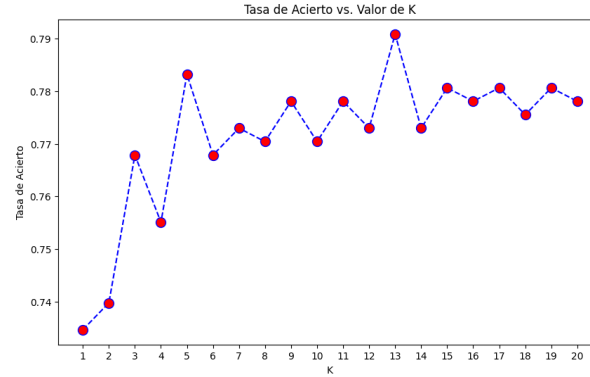


Figura 3: Curva de acierto en función del valor de K utilizando Hold-out Validation.

Con esta grafica podemos observar que el valor de K que maximiza la tasa de acierto es K=5 y K=13. Pero el valor mas cercano para probar es 5, por lo que lo empecé a realizar las pruebas y observar las metricas que arroja el modelo entrenado.

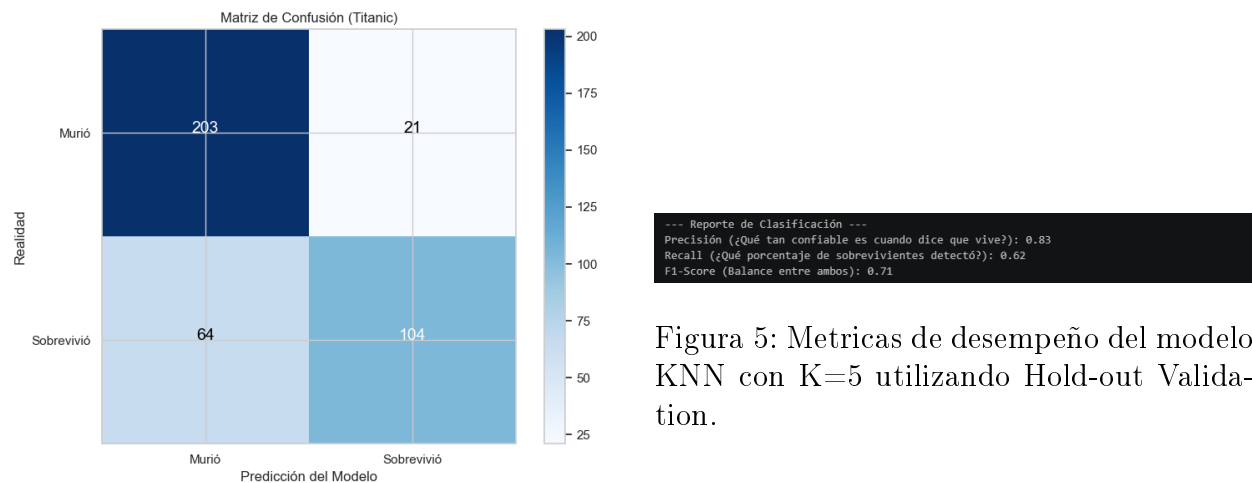


Figura 4: Matriz de confusión del modelo KNN con K=5 utilizando Hold-out Validation.

Por lo que se puede observar las metricas que nos arrojó fue la siguiente:

- Accuracy: 0.83
- Recall: 0.62
- F1-Score: 0.71

Y la distribución de los datos se ve de la siguiente manera:

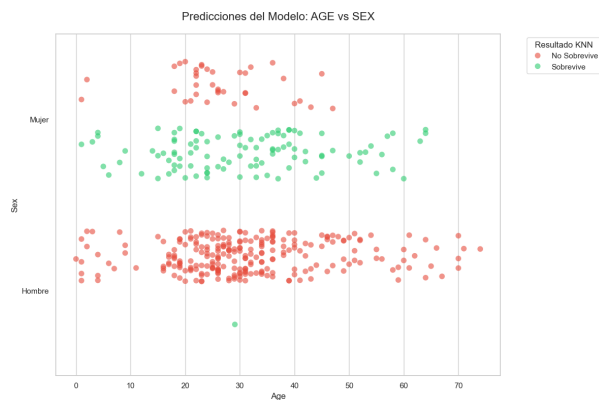


Figura 6: Predicciones del modelo KNN con $K=5$ utilizando Hold-out Validation.

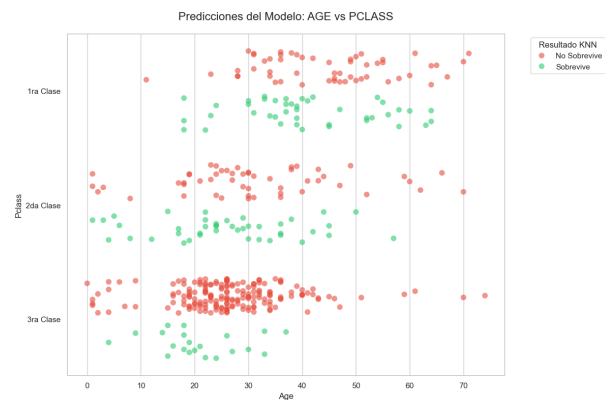


Figura 7: Distribución de predicciones del modelo KNN con $K=5$ utilizando Hold-out Validation.

Como se puede observar en la 6 y en la 7 el modelo tiene una buena capacidad para identificar a los sobrevivientes, especialmente en el grupo de mujeres y niños, lo cual es consistente con los patrones visuales observados inicialmente. Sin embargo, también se puede notar que el modelo tiene dificultades para clasificar correctamente a algunos hombres adultos, lo que sugiere que podría beneficiarse de un ajuste adicional o de la incorporación de características adicionales para mejorar su capacidad de discriminación. Por lo que ahora pasaremos a la segunda prueba, donde se implementara la validacion cruzada de K-pliegues para obtener una estimacion mas robusta del desempeño del modelo y asi poder comparar los resultados obtenidos con la tecnica de Hold-out.

4.2. Segunda prueba.- KNN y Validación Cruzada de K-Pliegues

En esta segunda prueba, se implementó la validación cruzada de K-pliegues para evaluar el desempeño del modelo KNN de manera más robusta. Se utilizó un valor de K igual a 5 para el clasificador y se dividió el dataset en 5 pliegues. El proceso se repitió iterativamente, rotando el pliegue de prueba en cada iteración, y se registraron las métricas de desempeño para cada pliegue.

```

Fold 1: Accuracy = 0.7780
Fold 2: Accuracy = 0.7661
Fold 3: Accuracy = 0.7683

---> Validación finalizada. Promedio: 0.7708
---> Mejor Accuracy detectado: 0.7780

=====

Fold 1: Accuracy = 0.7519
Fold 2: Accuracy = 0.8853
Fold 3: Accuracy = 0.8168
Fold 4: Accuracy = 0.7786
Fold 5: Accuracy = 0.7241

---> Validación finalizada. Promedio: 0.7754
---> Mejor Accuracy detectado: 0.8168

=====

Fold 1: Accuracy = 0.7647
Fold 2: Accuracy = 0.7754
Fold 3: Accuracy = 0.7887
Fold 4: Accuracy = 0.8128
Fold 5: Accuracy = 0.7861
...
Fold 10: Accuracy = 0.7538

---> Validación finalizada. Promedio: 0.7700
---> Mejor Accuracy detectado: 0.8244

```

Figura 8: Curva de acierto para obtener el valor optimo de K- Fold para realizar las pruebas.

Aqui podemos observar que el k optimo que se obtuvo fue de igual manera 5, este valor se usara para en el codigo de la lista (3).

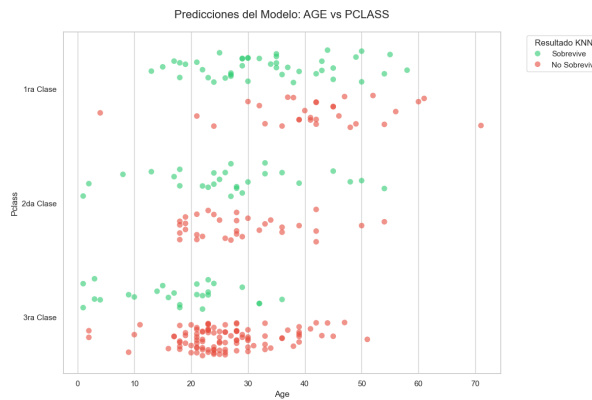


Figura 9: Prediccion del modelo KNN con K=5 utilizando Validación Cruzada de K-Pliegues. Edad vs Clase.



Figura 10: Predicciones del modelo KNN con K=5 utilizando Validación Cruzada de K-Pliegues.

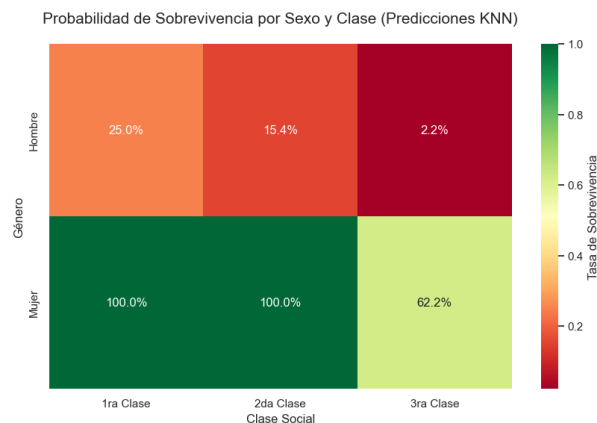


Figura 11: Probabilidad de supervivencia del modelo KNN con $K=5$ utilizando Validación Cruzada de K-Pliegues. Genero vs Clase.

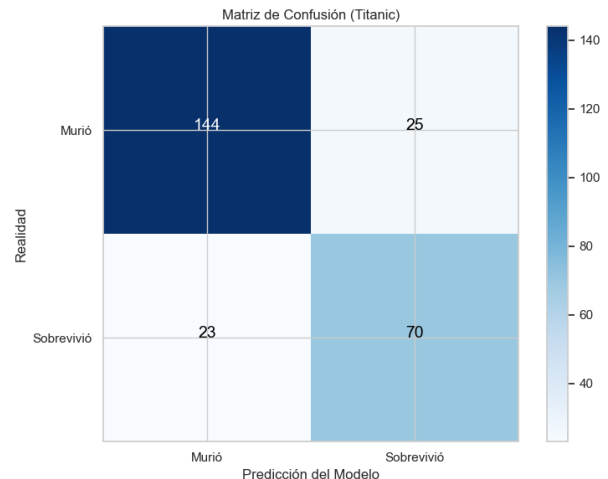


Figura 12: Matriz de Confusión del modelo KNN con $K=5$ utilizando Validación Cruzada de K-Pliegues.

Este mapa de calor nos da una visualización clara de cómo el modelo KNN con validación cruzada ha aprendido a asociar ciertas características con la probabilidad de supervivencia. Podemos observar que las mujeres de clase alta tienen una probabilidad significativamente mayor de sobrevivir, lo cual es consistente con los patrones históricos del desastre del Titanic. Por otro lado, los hombres adultos de clase baja presentan una probabilidad mucho menor de supervivencia, lo que también refleja la realidad histórica y valida la capacidad del modelo para capturar estas relaciones complejas entre las características de los pasajeros y su destino final.

5. Conclusión

En este estudio se implementó un clasificador de K-Vecinos más Cercanos (KNN) para predecir la supervivencia de los pasajeros del Titanic, utilizando tanto la técnica de Hold-out como la validación cruzada de K-pliegues para evaluar su desempeño. Los resultados obtenidos mostraron que el modelo KNN con $K = 5$ logró una tasa de acierto significativa, especialmente al identificar a mujeres y niños como sobrevivientes, lo cual es coherente con los patrones históricos del desastre. También demuestra de forma muy cruda la realidad de la toma de decisiones en situaciones de emergencia, donde factores como el género y la clase social jugaron un papel crucial en la supervivencia de los pasajeros. Por lo tanto, este análisis no solo valida la eficacia del modelo KNN para esta tarea de clasificación, sino que también proporciona una perspectiva histórica sobre las dinámicas sociales presentes en el contexto del Titanic.

También aprendí bastante en el desarrollo de este proyecto, desde la implementación de los modelos, como también la forma en la que se manejan los datos de entrada, ya que viví la experiencia de que al principio el modelo al momento de ingresar los datos de forma incorrecta las métricas de evaluación que regresaba eran muy bajas por lo que al momento de realizar la limpieza y normalización de los datos el modelo mejoró significativamente, esto me

hizo entender la importancia de tener un buen preprocesamiento de los datos para obtener buenos resultados en el modelo. Y tambien me di cuenta de la importancia de la validacion cruzada para obtener una estimacion mas robusta del desempeño del modelo, ya que al realizar la validacion cruzada se obtiene una mejor idea de como el modelo se comporta con diferentes particiones de los datos, lo que permite identificar si el modelo esta sobreajustado o subajustado. En resumen, este proyecto no solo permitió aplicar técnicas de machine learning para resolver un problema de clasificación, sino que también proporcionó una comprensión más profunda de la importancia del preprocesamiento de datos y la validación cruzada en el desarrollo de modelos predictivos efectivos.

Bibliográficas

- [1] Christopher M Bishop. *Pattern recognition and machine learning*. Springer, 2006.
- [2] Tom Fawcett. «An introduction to ROC analysis». En: *Pattern recognition letters* 27.8 (2006), págs. 861-874.
- [3] Dr. Marco Antonio Aceves Fernandez. *De 0 a 100 en Inteligencia Artificial. Todas las claves para comenzar tu viaje por la IA*. 2025. Cap. 4, págs. 252-354.
- [4] Trevor Hastie, Robert Tibshirani y Jerome Friedman. *The elements of statistical learning: data mining, inference, and prediction*. Springer Science & Business Media, 2009.
- [5] Gareth James et al. *An introduction to statistical learning*. Springer, 2013.
- [6] Nathalie Japkowicz y Mohak Shah. *Evaluating learning algorithms: a classification perspective*. Cambridge University Press, 2011.
- [7] Sebastian Raschka. *Python machine learning*. Packt Publishing Ltd, 2015.
- [8] Shai Shalev-Shwartz y Shai Ben-David. *Understanding machine learning: From theory to algorithms*. Cambridge university press, 2014.